# SQL TABLE RELATIONSHIPS

Complete Learning Guide: Definitions • Conditions • Rules • Diagrams • Examples • MySQL Code • Practice

## What is a table relationship?

A table relationship connects related rows in two or more database tables. SQL uses keys—especially Primary Keys (PK) and Foreign Keys (FK)—to maintain valid links and reduce duplicated data.

## The Four Relationship Types

Many introductory database courses teach three main cardinalities (1:1, 1:N, M:N). Some courses list four types by separating One-to-Many and Many-to-One. This guide covers all four commonly requested labels: One-to-One, One-to-Many, Many-to-One, and Many-to-Many.

| Type | Meaning | Example |
|---|---|---|
| One-to-One (1:1) | One row matches at most one row on the other side. | Person ↔ Passport |
| One-to-Many (1:N) | One parent row can match many child rows. | Department → Employees |
| Many-to-One (N:1) | Many rows in one table point to one row in another. | Employees → Department |
| Many-to-Many (M:N) | Many rows on each side can relate to many on the other. | Students ↔ Courses |

# 1. One-to-One Relationship (1:1)

**Definition:** Each person can have at most one passport record in this design, and each passport record belongs to one person.

## Conditions and rules

• A parent table has a Primary Key. • The related table has a Foreign Key. • The Foreign Key must also be UNIQUE or itself be the child table's Primary Key. • Use NOT NULL if every child row must point to a parent. • A Foreign Key + UNIQUE enforces at most one matching child row per parent; it does not force every parent to have a child row.

**One-to-One: Person ↔ Passport**

Each person_id appears no more than once in Passport.

## Example data

| Person.person_id (PK) | person_name | Passport.person_id (FK, UNIQUE) | passport_number |
|---|---|---|---|
| 101 | Ravi | 101 | P12345 |
| 102 | Priya | 102 | P12346 |
| 103 | Anjali | 103 | P12347 |

## MySQL example

```
CREATE TABLE Person (
    person_id INT PRIMARY KEY,
    person_name VARCHAR(50)
);

CREATE TABLE Passport (
    passport_id INT PRIMARY KEY,
    person_id INT NOT NULL UNIQUE,
    passport_number VARCHAR(20),
    FOREIGN KEY (person_id) REFERENCES Person(person_id)
);
```

**How to identify it:** Check that the child FK is unique and references the parent key. A duplicate FK value in Passport should be rejected.

**Common use cases:** Person–Passport, User–Profile, Employee–LockerAssignment (depending on business rules).

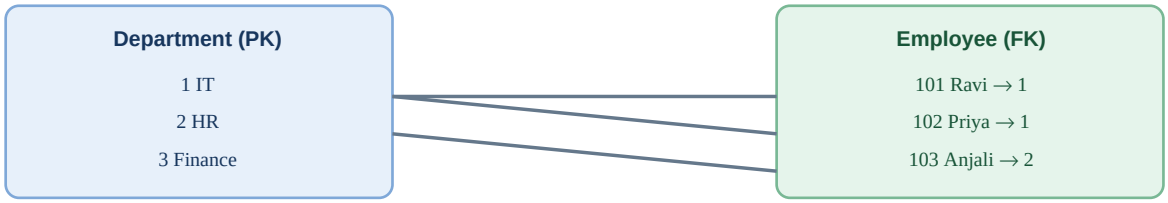
# 2. One-to-Many Relationship (1:N)

**Definition:** One department can have many employees, while each employee is assigned to at most one department in this design.

## Conditions and rules

• Parent table has a Primary Key. • Child table contains the Foreign Key. • The FK can repeat in the child table. • Do not add UNIQUE to the FK if multiple child rows must point to the same parent. • A nullable FK permits an employee to have no department; use NOT NULL if assignment is mandatory.

**One-to-Many: Department → Employees**




Several employees can reference the same department_id.

## Example data

| Department.department_id (PK) | department_name | Employee.employee_id (PK) | department_id (FK) |
|---|---|---|---|
| 1 | IT | 101 Ravi | 1 |
| 1 | IT | 102 Priya | 1 |
| 2 | HR | 103 Anjali | 2 |

## MySQL example

```
CREATE TABLE Department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50)
);

CREATE TABLE Employee (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(50),
    department_id INT,
    FOREIGN KEY (department_id)
        REFERENCES Department(department_id)
);
```

**How to identify it:** The parent key is unique; the FK column in the child table may contain repeated values.

**Common use cases:** Department–Employees, Customer–Orders, Author–Books (if each book has one author).

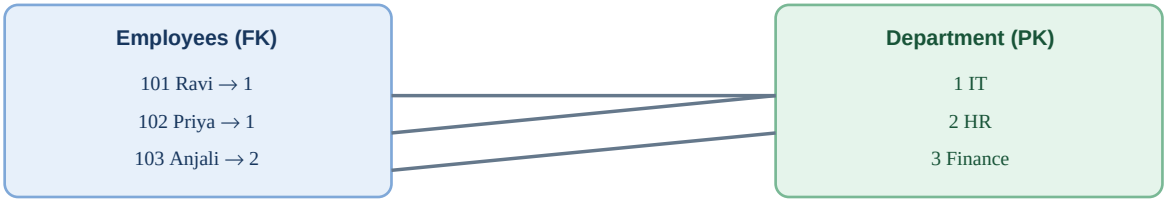
# 3. Many-to-One Relationship (N:1)

**Definition:** Many employee rows can refer to one department row. This is the same relationship as One-to-Many, viewed from the Employee table's side.

## Conditions and rules

• The many-side table contains the FK. • Multiple rows in the many-side table may share the same FK value. • The referenced department key must be unique. • The FK may be nullable unless NOT NULL is added. • The relationship is N:1 from Employee to Department, but 1:N from Department to Employee.

**Many-to-One: Employees → Department**




Several employee rows point to the same department row.

## Example data

| Employee.employee_id (PK) | employee_name | department_id (FK) | Department.department_name |
|---|---|---|---|
| 101 | Ravi | 1 | IT |
| 102 | Priya | 1 | IT |
| 103 | Anjali | 2 | HR |

## SQL to view the many-to-one mapping

```
SELECT e.employee_name, d.department_name
FROM Employee AS e
JOIN Department AS d
  ON e.department_id = d.department_id;
```

**How to identify it:** Look from the child/many-side table. If many employee rows can reference the same department, the Employee-to-Department direction is N:1.

**Important:** Many-to-One and One-to-Many are not different physical designs; they describe the same link from opposite directions.

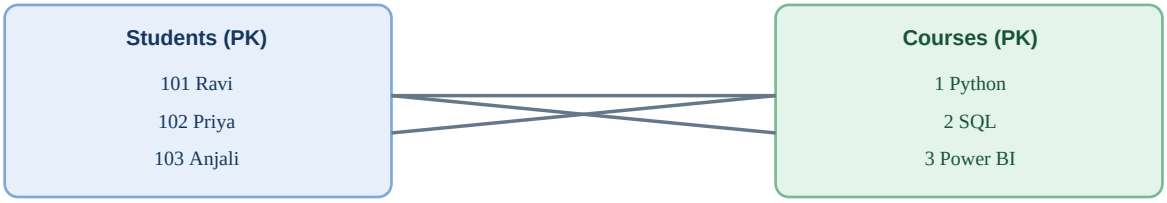
# 4. Many-to-Many Relationship (M:N)

**Definition:** Many students can enroll in many courses. One student may have several courses, and one course may have several students.

## Conditions and rules

• Create two main tables, each with a Primary Key. • Create a junction/bridge table. • The bridge table contains a Foreign Key to each main table. • A composite Primary Key (student_id, course_id) prevents the same student-course pair from being inserted twice. • Each FK can repeat individually.

**Many-to-Many: Students ↔ Courses**




The Enrollments bridge table stores the links.

## Bridge table example

| Enrollments.student_id (FK) | Enrollments.course_id (FK) | Meaning |
|---|---|---|
| 101 | 1 | Ravi studies Python |
| 101 | 2 | Ravi studies SQL |
| 102 | 1 | Priya studies Python |
| 103 | 3 | Anjali studies Power BI |

## MySQL example

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50)
);
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50)
);
CREATE TABLE Enrollments (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

**How to identify it:** A student can appear in multiple enrollment rows and a course can appear in multiple enrollment rows. The junction table stores each pair.

**Common use cases:** Students–Courses, Products–Orders (if an order can contain many products and a product can appear in many orders).

# Quick Comparison and Practice

| Feature | One-to-One | One-to-Many | Many-to-One | Many-to-Many |
|---|---|---|---|---|
| Notation | 1:1 | 1:N | N:1 | M:N |
| FK location | Related/child table | Child/many side | Employee/many side | Bridge table |
| FK repeats? | No, if UNIQUE/PK | Yes | Yes | Yes individually |
| Bridge table? | Usually no | No | No | Yes |
| Example | Person–Passport | Department–Employees | Employees–Department | Students–Courses |

## General SQL rules to remember

1. A Primary Key uniquely identifies each row and cannot be NULL. 2. A Foreign Key must match an existing referenced key unless it is NULL and NULL is allowed. 3. A Foreign Key can be NULL unless NOT NULL is specified. 4. To enforce 1:1, add UNIQUE to the FK or use the FK as the child PK. 5. In 1:N, the FK is usually not unique. 6. In M:N, use a bridge table and usually a composite PK. 7. The referenced and referencing data types must be compatible. 8. ON DELETE CASCADE is optional and should be used only when child rows should be deleted with the parent.

## SQL queries for checking duplicates

```
-- Check duplicate employee department IDs (repeats are allowed in 1:N)
SELECT department_id, COUNT(*) AS total
FROM Employee
GROUP BY department_id
HAVING COUNT(*) > 1;

-- Check duplicate person IDs in Passport (should return no rows for 1:1)
SELECT person_id, COUNT(*) AS total
FROM Passport
GROUP BY person_id
HAVING COUNT(*) > 1;

-- Inspect constraints in MySQL
SHOW CREATE TABLE Passport;
```

Note: Data checks alone do not prove a relationship. Check Foreign Key and UNIQUE/Primary Key constraints too.

## Practice questions

1. One person has one passport. Which type? **Answer: One-to-One.**
2. One department has many employees. From Department to Employee? **One-to-Many.**
3. Many employees point to one department. From Employee to Department? **Many-to-One.**
4. Many students take many courses. Which type and table? **Many-to-Many using Enrollments.**